



兰州理工大学

LANZHOU UNIVERSITY OF TECHNOLOGY

毕业设计

题目 基于用户行为分析的实时信息流排序与分发系统

学生姓名 甄弘硕

学 号 220162701059

专业班级 22 软件工程 1 班

指导教师 韦慧怡,

学 院 计算机与人工智能学院

答辩日期 2026 年 6 月 日

基于用户行为分析的实时信息流排序与分发系统
Real-time Information Flow Ranking and Distribution System Based on User
Behavior Analysis

甄弘硕(ZHEN hongShuo)
220162701059

兰州理工大学毕业设计

摘要

随着社交媒体内容规模持续增长，单纯按发布时间倒序展示的信息流已难以满足用户的个性化阅读需求。本文围绕“基于用户行为分析的实时信息流排序与分发系统”这一课题，设计并实现了一套类 X/Twitter 的社交媒体推荐原型系统。系统采用前后端分离架构，后端基于 Spring Boot 提供 RESTful API 与 WebSocket 服务，前端基于 Vue 3 构建单页应用，数据库采用 MySQL 存储用户、内容、行为、关注、通知、私信、广告等业务数据。系统实现了注册登录、内容发布、评论、点赞、点踩、转发、引用、关注、搜索、实时通知、私信、用户画像、广告投放和后台管理等功能。

在推荐算法方面，系统通过行为记录表（Behavior）统一记录浏览、点赞、评论、转发、搜索、点踩和快速滑过等行为事件，结合停留时长、内容标签与社交关系构建用户画像。在排序阶段，系统融合多行为加权评分、TF-IDF 内容相似度、协同过滤、时间衰减、热门话题加成、负反馈过滤、作者多样性惩罚和随机探索等策略，形成可解释的混合推荐结果。同时，系统接入 Ollama 本地大模型作为 AI 推荐策略，并通过策略管理器支持管理员在传统多因子推荐与 AI 推荐之间切换。为便于验证算法效果，系统还提供推荐流与时间流对比、权重调节、排序管道漏斗图 and 用户画像可视化等功能。

关键词：信息流推荐；用户行为分析；用户画像；混合排序；Spring Boot

兰州理工大学毕业设计

Abstract

With the rapid growth of social media content, traditional chronological feeds can no longer satisfy users' personalized reading needs. This project designs and implements a real-time feed ranking and distribution system based on user behavior analysis. The system adopts a front-end and back-end separated architecture. The back end is built with Spring Boot and provides RESTful APIs, while the front end is implemented as a Vue 3 single-page application. MySQL is used to store users, posts, behaviors, relationships, notifications, advertisements and other business data.

The system supports user authentication, post publishing, comments, likes, dislikes, reposts, quotes, following, search, real-time notifications, private messages, user personas, advertisement delivery and administrator management. For recommendation, the system records behavior events such as views, likes, comments, reposts, searches, dislikes and quick skips, builds user interest profiles, and combines multi-behavior weighted scoring, TF-IDF content similarity, collaborative filtering, time decay, trending topic boosting, negative feedback filtering, author diversity penalty and controlled exploration. In addition, an Ollama-based AI recommendation strategy is integrated, and administrators can switch between the traditional hybrid strategy and the AI strategy. The system also provides recommendation comparison, parameter tuning, pipeline funnel visualization and user persona visualization for demonstration and evaluation.

Key Words: feed recommendation; user behavior analysis; user persona; hybrid ranking; Spring Boot

兰州理工大学毕业设计

目 录

摘 要.....	3
第 1 章 绪论.....	7
1.1 研究背景及意义.....	7
1.2 设计目的及主要内容.....	8
1.3 开发技术介绍.....	9
1.3.1 Spring Boot 技术.....	9
1.3.2 Vue 3 技术.....	9
1.3.3 MySQL、Redis 与 WebSocket 技术.....	10
1.3.4 推荐算法与 AI 技术.....	10
1.3.5 技术选型理由与替代方案.....	11
1.4 国内外研究现状.....	11
1.5 本文组织结构.....	12
第 2 章 系统需求分析.....	12
2.1 需求分析.....	12
2.2 可行性分析.....	14
2.3 业务流程分析.....	15
2.4 数据流与数据字典.....	16
2.5 非功能需求分析.....	17
2.6 需求与实现对应关系.....	17
第 3 章 系统总体设计.....	18
3.1 系统总体架构设计.....	18
3.2 功能模块设计.....	19
3.3 数据库设计.....	20
3.4 推荐算法设计.....	24
3.5 用户画像设计.....	28
3.6 安全与实时通信设计.....	29
3.7 广告分发设计.....	29
3.8 算法可视化设计.....	30
3.9 核心 REST 接口与 DTO 约定.....	30
3.10 关键术语与符号说明.....	31

兰州理工大学毕业设计

3.11 推荐流水线与复杂度分析	32
第 4 章 系统详细设计及实现	33
4.1 开发环境与项目结构	34
4.2 部署配置与运维要点	35
4.3 用户认证与内容互动实现	36
4.4 推荐排序实现	37
4.5 AI 模块与管理后台实现	38
4.6 广告智能分发实现	39
第 5 章 系统测试及使用说明	40
5.1 系统测试报告	40
5.1.1 测试的目的及原则	40
5.1.2 测试过程及结果	40
5.1.3 测试总结	43
5.1.4 测试数据构造与实验建议	44
5.2 系统使用说明	44
5.2.1 系统简介	44
5.2.2 系统运行环境	45
5.2.3 系统操作说明	45
设计总结	46
工作总结	46
不足与展望	47
个人收获	48
系统功能验证流程概要	48
参考文献	48
外文翻译	50
原文	50
译文	50
附录 程序源代码	50
致 谢	50

第1章 绪论

1.1 研究背景及意义

随着移动互联网和社交媒体平台的快速发展，用户每天面对的信息规模持续扩大。以微博、X、抖音、B 站等平台为代表的內容社区，已经从单纯的信息发布工具发展为內容消费、社交互动、广告分发和智能服务的综合平台。在这样的场景下，平台內容供给远远超过用户能够主动浏览和筛选的范围，信息过载成为影响用户体验的重要问题。

信息过载不仅意味着“看不完”，还会导致注意力资源的稀缺分配：用户在同一停留时间内只能消费有限条內容，平台必须在海量候选中做出取舍。传统时间流假设“越新越重要”，隐含前提是用户对所有发布者具有均匀关注度，这与真实社交网络中的兴趣差异、互动强度和內容偏好并不一致。推荐排序试图依据浏览、点赞、评论、关注等行为推断用户偏好，使每一次刷新更具针对性，从而提升用户满意度与平台粘性，这与基于用户浏览行为的个性化推荐研究以及深度学习推荐系统的基本目标一致[7][11]。

传统信息流通常采用发布时间倒序排列方式，即最新发布的内容排在最前面。这种方式实现简单，能够保证时效性，但缺少对用户兴趣、內容质量、社交关系和行为反馈的综合考虑。当平台內容数量增加后，用户需要花费大量时间从时间流中寻找有价值內容，容易错过真正感兴趣的帖子。同时，优质內容也可能因为发布时间较早而快速失去曝光机会，平台內容分发效率受到限制。

推荐排序技术为解决上述问题提供了有效途径。系统可以通过采集用户浏览、点赞、评论、转发、搜索、关注、点踩、快速滑过等多维行为，分析用户兴趣偏好和互动习惯，并结合內容标签、文本相似度、热门话题、社交关系和时间衰减等因素，对信息流进行个性化排序。相比单一时间倒序，个性化推荐能够在保持內容新鲜度的同时提高兴趣命中率，使用户更快获得符合自身需求的信息。

本课题以类 X/Twitter 社交媒体平台为业务背景，设计并实现一套基于用户行为分析的实时信息流排序与分发系统。系统既包含注册登录、发帖、评论、点赞、转发、引用、关注、搜索、通知、私信等社交媒体基础功能，也重点实现用户画像、混合多因子推荐、协同过滤、TF-IDF 內容相似度、AI 推荐策略、算法效果对比和信息流广告智能分发等核

兰州理工大学毕业设计

心模块。通过该课题的设计与实现，可以综合训练 Web 系统开发、数据库建模、推荐算法工程化、实时通信、数据可视化和 AI 服务集成等能力。

1.2 设计目的及主要内容

本设计的主要目的是构建一个具备完整业务链路和推荐排序能力的社交媒体信息流系统。系统需要在前后端分离架构下实现用户端、管理员端和算法验证端，并通过统一的数据模型将用户行为、内容特征、社交关系和推荐结果连接起来，形成可验证、可测试、可扩展的软件系统。

本设计的总体目标是在项目周期内交付一套可运行、可演示、可验证的社交媒体信息流推荐系统，并形成覆盖需求分析、系统设计、编码实现和测试验证的技术文档。功能层面，系统需要贯通用户认证、内容生产、社交互动、实时通知、私信、搜索、广告与后台治理等链路；算法层面，需要实现以用户行为画像为核心的混合排序，并结合 TF-IDF、协同过滤和 AI 推荐策略进行对照；工程层面，需要保持后端分层清晰、前端组件职责明确、接口命名稳定，并在 AI 推理、数据导入等长耗时场景中提供降级或补齐机制；评测层面，需要通过对比实验、评分明细、漏斗图和参数调节面板展示推荐策略的有效性。

本文所称“实时”主要指交互层面的及时反馈，例如通知推送、私信到达、信息流刷新和策略切换后的结果更新，而非面向超高并发生产环境的极限吞吐指标。传统混合推荐路径以毫秒级到秒级响应为目标；AI 推荐路径由于依赖本地大模型推理，采用缓存优先与异步预计算思路，在缓存未命中时先返回传统推荐结果，同时后台触发 AI 排序计算，以保证信息流主链路可用。

本课题的主要内容包括以下几个方面。第一，设计并实现系统基础平台，包括用户注册登录、角色管理、内容发布、图片上传、评论与多级回复、点赞、点踩、转发、引用、关注、搜索、热门话题、通知和私信等功能。第二，设计统一的用户行为数据模型，记录浏览、点赞、评论、转发、搜索、点踩、快速滑过等行为，为用户画像和推荐算法提供数据基础。第三，构建用户画像模块，从兴趣标签、作者偏好、行为强度、互动风格、内容深度偏好和新鲜度偏好等维度描述用户。第四，实现混合多因子推荐排序算法，

兰州理工大学毕业设计

将多行为加权评分、互动率、关注来源加成、标签匹配、TF-IDF 相似度、协同过滤、热门话题加成、时间衰减、负反馈过滤、作者多样性惩罚和随机探索机制结合起来。第五，集成 AI 大语言模型，提供 AI 自动打标、AI 推荐和智能问答能力。第六，实现广告智能分发模块，根据用户兴趣标签、广告标签、出价和历史点击率计算广告匹配分，并按配置在信息流中插入原生广告。第七，实现算法效果可视化对比，包括推荐流与时间流对比、多用户画像对比、推荐评分详情、参数调节面板和排序管道漏斗图。

通过上述设计，系统能够较完整地覆盖从内容产生、行为采集、画像构建、候选召回、排序分发到效果验证的推荐链路。该系统在软件工程完整性与推荐算法应用价值之间形成相互支撑的整体。

1.3 开发技术介绍

1.3.1 Spring Boot 技术

Spring Boot 是基于 Spring 生态的快速开发框架，具有自动配置、依赖管理、嵌入式服务器和工程结构清晰等特点。后端系统采用 Spring Boot 3.2.1 构建 RESTful API，按 controller、service、repository、model、dto 和 config 等层次组织代码。控制层负责接收前端请求并返回统一响应，业务层负责处理推荐、画像、广告、通知和内容互动逻辑，数据访问层基于 Spring Data JPA 操作 MySQL 数据库。

在本系统中，Spring Boot 还承担 WebSocket 通信、文件上传映射、跨域配置、AI 服务调用和管理员接口管理等任务。通过框架提供的注解机制和依赖注入机制，系统各模块之间保持较低耦合，便于后续扩展新的推荐策略或业务功能。

1.3.2 Vue 3 技术

Vue 3 是渐进式 JavaScript 前端框架，适合构建单页应用。系统前端使用 Vue 3 与 Vite 搭建，结合 Element Plus 实现页面组件，使用 Axios 与后端进行 REST API 通信，使用 ECharts 展示用户画像、广告报表和推荐效果对比等图表。前端页面主要包括首页信息流、登录注册、个人主页、搜索页、通知页、私信页、管理后台、算法验证页、广告报表页和 AI 助手页。

兰州理工大学毕业设计

Vue 组件化开发方式使页面结构清晰，信息流列表组件（FeedList）、帖子卡片组件（TweetCard）、右侧面板组件（RightPanel）、权重调节组件（WeightTuner）等可以复用在不同场景中。对于推荐系统而言，前端不仅负责展示帖子列表，还需要呈现推荐理由、评分详情、当前推荐策略、广告卡片和算法对比指标，因此数据交互和可视化能力尤为重要。

1.3.3 MySQL、Redis 与 WebSocket 技术

MySQL 是本系统的主要持久化数据库，用于存储用户、帖子、行为、关注关系、标签、通知、私信、广告和广告配置等数据。关系型数据库适合表达系统中的实体关系，例如用户与内容、用户与关注、内容与标签、内容与评论之间的关联。系统通过 JPA 实体和 Repository 层完成数据访问，减少手写 SQL 的工作量。

Redis 可用于缓存和临时状态管理，在推荐系统中可扩展用于热门话题缓存、用户画像缓存和接口限流等场景。WebSocket 用于实现通知和私信的实时推送。后端通过 STOMP 用户队列向指定用户推送消息，前端建立连接后订阅相应通道，使点赞、评论、关注和私信等事件能够及时展示。

1.3.4 推荐算法与 AI 技术

推荐排序是本系统的核心技术。系统采用混合多因子推荐策略，将内容互动分、用户画像匹配、文本相似度、协同过滤、热门话题、新鲜度、负反馈和随机探索等因素组合为综合得分。TF-IDF 用于衡量用户历史兴趣文本与候选内容之间的相似度，协同过滤用于根据相似用户行为发现潜在兴趣内容，时间衰减用于平衡新鲜内容和长尾内容的曝光机会。

在 AI 集成方面，系统使用 Ollama 本地模型完成内容打标和 AI 推荐，并使用 DeepSeek API 支持智能问答助手。AI 推荐策略通过用户画像和候选帖子摘要构造提示词，由大语言模型输出排序结果与推荐理由；策略管理器在此基础上加入缓存和异步预计算机制，使 AI 能力在不阻塞主信息流的前提下参与排序。管理员可在后台切换传统多因子推荐与 AI 推荐策略，以便对比两种排序机制的差异。

1.3.5 技术选型理由与替代方案

后端选用 Spring Boot，主要考虑其 Java 生态成熟、RESTful API 开发成本较低，且与任务书提出的前后端分离 B/S 架构相匹配[6][9]。前端选用 Vue 3，是因为其组合式 API、组件化组织和管理后台生态较适合表单密集型页面与可视化页面开发[10]。持久层选用 MySQL，能够满足本文用户、帖子、互动、消息、通知、广告与推荐日志等结构化数据的事务一致性要求；若后续扩展到更大规模日志分析，可再引入搜索引擎或数据仓库组件[13]。本地推理选用 Ollama 主要用于 AI 推荐解释与候选重排的辅助实验，核心排序仍保留 Hybrid 策略作为稳定降级路径，便于在工程实现中兼顾智能化与可维护性[11][14]。

1.4 国内外研究现状

工业界推荐系统已从早期协同过滤与矩阵分解，逐步演进为融合深度模型、多任务学习与海量日志训练的排序架构。YouTube 推荐系统相关研究表明，深度神经网络可以综合用户历史行为、上下文和候选内容特征，从而服务于大规模视频推荐场景[21]。X 平台开源的推荐算法也展示了候选召回、过滤与重排序等工程流水线思想，为本课题“关注源 + 全站源”的双源候选组织提供了现实参照[15]。

在国内学术语境下，信息流推荐、用户画像和算法治理同样是研究热点。已有研究从科技传播、用户浏览行为、用户画像构建和推荐去偏等角度指出，推荐系统既要提升内容匹配效率，也要兼顾用户兴趣漂移、信息茧房和结果可解释性等问题[1][7][8][20]。因此，本文在实现混合推荐排序时，不只追求点击与互动分数，还加入作者多样性、负反馈衰减、冷启动探索和评分拆分展示，使算法逻辑能够对应系统实际功能与可视化分析要求。

近年来，深度学习推荐模型和 CTR 预估模型不断强化对用户兴趣序列、交叉特征和上下文信息的建模能力，DeepFM、DIEN 等方法说明了深度模型在点击率预测与兴趣演化建模中的价值[18][19]。本文系统没有直接训练大规模深度推荐模型，而是在可解释混合打分基础上增加 AI 推荐策略，用于候选理解、排序解释和降级对比，从而在毕业设计规模的数据条件下体现智能推荐思想[11][14]。

矩阵分解和深度推荐模型通常通过低维向量表示用户与物品，并利用历史交互学习

兰州理工大学毕业设计

潜在偏好，能够在大规模行为日志中发现人工标签难以覆盖的兴趣关系[16][18][19]。与这类需要持续训练和较大数据规模的方法相比，本文当前实现更强调可运行、可解释和易调参的多因子启发式排序；协同过滤加成、文本相似度加成和动态权重共同构成了适合本系统数据规模的折中方案。

在国内工程语境中，Spring Boot、Vue 与 MySQL 的组合具有较低学习成本和较好的资料基础，适合完成任务书要求的前后端分离信息系统[6][9][10]。数据库层面，本文以关系型模型承载用户、帖子、互动、消息、通知、广告和推荐日志等核心对象，并通过索引与业务分层控制查询复杂度[13]。算法层面，矩阵分解、协同过滤和深度学习推荐系统的理论为本文理解用户兴趣建模、隐式反馈和排序评估提供了方法参照[11][16][17]。

接口设计方面，系统围绕用户、帖子、互动、关注、消息、通知、广告、画像和推荐策略等资源划分 RESTful API，使前端页面能够通过统一路径访问后端服务；实时私信和通知则通过消息通道实现近实时推送。该设计符合任务书对 Spring Boot RESTful API、Vue 单页应用和前后端分离 B/S 架构的要求，也便于后续进行接口测试、权限控制和功能扩展[6][13]。

1.5 本文组织结构

全文包含中文摘要与关键词、英文 Abstract 与 Key Words。正文依照软件工程生命周期组织：第 2 章给出需求、可行性、业务流程与数据字典；第 3 章给出总体架构、模块设计、数据库与核心算法（混合推荐、画像、广告与可视化），并在章末给出术语与符号说明及推荐流水线与复杂度分析（3.10、3.11 节）；第 4 章描述开发环境、部署运维、关键模块实现、测试用例、测试结果分析及测试数据构造（含 4.2、4.9 节）；第 5 章总结工作、分析不足与展望，并给出系统功能验证流程概要（5.4 节）。文后为参考文献与致谢。

第 2 章 系统需求分析

2.1 需求分析

根据课题任务要求和系统业务场景，系统面向两类主要角色：普通用户和管理员。普通用户主要完成内容消费与社交互动，包括注册登录、浏览推荐信息流、发布帖子、

兰州理工大学毕业设计

上传图片、评论、点赞、点踩、转发、引用、关注、取关、搜索内容、查看通知、发送私信和查看个人画像等操作。管理员主要完成平台治理与运维配置，包括用户管理、角色切换、封禁用户、平台统计、推荐策略切换、AI 批量打标、Kaggle 数据导入、广告管理和算法验证等操作。

表 2-1 系统功能需求表

功能模块	主要需求
用户模块	注册、登录、用户资料查看与编辑、头像上传、角色管理、封禁管理。
内容模块	发帖、图片上传、评论、多级回复、转发、引用、删除、帖子详情查看。
社交模块	关注、取关、关注状态查询、关注列表、粉丝列表和推荐关注。
行为模块	记录浏览、点赞、评论、转发、搜索、点踩、跳过等行为事件。
推荐模块	个性化推荐、关注流、传统推荐策略、AI 推荐策略和策略热切换。
画像模块	兴趣标签、行为统计、活跃时段、用户类型、内容偏好、词云和雷达图。
搜索模块	综合搜索、帖子搜索、用户搜索、话题搜索和搜索建议。
通知私信模块	点赞、评论、关注、转发、引用通知，未读数管理和实时私信。
广告模块	信息流广告插入、广告匹配、广告创建编辑、展示点击统计。
管理模块	用户管理、平台统计、推荐策略管理、Kaggle 数据导入、AI 打标和广告管理。

上表按模块聚合需求，可与课题需求条目逐项对照；实际实现时仍存在跨模块耦合点需要注意。例如推荐模块依赖行为模块的完备性——若搜索行为（SEARCH）未记录，则画像缺少主动意图信号；广告模块依赖画像模块的标签质量——若 AI 打标失败则匹配度下降；通知模块依赖内容模块的评论层级正确性——若父内容字段（parentContent）关联错误，将导致通知定位失败。职责划分并不能取代集成测试，第 4 章据此给出端到端场景用例。

系统的非功能需求主要包括可用性、可维护性、可扩展性、实时性、安全性和可视化能力。可用性要求普通用户能够按照常见社交媒体操作习惯完成浏览与互动；可维护性要求后端分层清晰、前端组件职责明确；可扩展性要求推荐策略可替换、广告策略可

兰州理工大学毕业设计

扩展；实时性要求通知和私信能够及时推送；安全性要求登录状态与角色权限具备基本控制；可视化能力要求推荐效果和用户画像能够通过图表进行展示。

普通用户视角：需要稳定登录与会话保持、清晰的信息流切换（For You / Following）、可靠的互动反馈（计数与通知）。管理员视角：需要便捷的用户治理、策略切换与外部数据导入能力。推荐与画像视角：需要统一 behavior 模型与足够的日志字段支撑离线分析与在线排序。广告视角：需要定向字段与统计闭环。AI 视角：需要可配置模型地址与超时降级。

普通用户侧细化：除发帖与互动外，需保证会话失效后可重新登录且不丢失历史行为沉淀；推荐流与关注流切换时应维持分页或刷新策略一致，避免误跳。私信侧需支持未读状态与送达感知。

管理员侧细化：后台操作应具备明确反馈与列表刷新；批量导入与批量打标属于长耗时任务，理想情况下可进一步异步化并以任务 ID 查询进度；当前实现通过限制单次导入规模和失败时使用内置内容库补齐，保证演示数据可以稳定生成。

推荐子系统：需暴露策略类型与评分明细字段以便算法验证页展示；候选为空时应返回空列表而非错误码，以便前端区分“无内容”与“接口故障”。

广告子系统：需区分广告启用与全局插入开关；无可用广告时不应阻断信息流主体加载。

AI 子系统：需区分打标模型与推荐模型的配置键；云端对话与本地推理失败时应互不牵连。

2.2 可行性分析

从技术可行性看，系统采用的技术均较成熟。Spring Boot 能够支撑后端 REST API 开发和业务模块组织；Vue 3 能够完成前端单页应用开发；MySQL 能够满足本文数据规模下的存储要求；WebSocket 能够实现实时通知和私信；ECharts 能够完成算法效果、画像和广告报表可视化；Ollama 和 DeepSeek API 能够为 AI 推荐、AI 打标和智能助手提供模型能力。因此，系统在技术上具有可实现性。

从经济可行性看，系统主要依赖开源框架和本地开发环境。Spring Boot、Vue、

兰州理工大学毕业设计

MySQL、ECharts、Maven、Node.js 等工具均可免费使用，AI 推荐可以通过本地 Ollama 模型运行，降低外部接口调用成本。在本项目约束下，开发与部署成本可控。

从操作可行性看，系统界面参照主流社交媒体交互方式设计。普通用户能够通过首页信息流完成浏览、发帖、点赞、评论和关注等常见操作，管理员能够在后台完成用户管理、推荐策略切换、AI 打标和广告管理。系统操作流程符合常见 Web 应用习惯，具备可操作性。

数据可行性：系统可通过 DataSeeder、Kaggle 公开数据集导入和内置内容库补齐构造多标签、多用户、多行为样本，满足算法验证所需的最小数据规模。风险可行性：主要风险包括外部数据源不可用、本地模型资源占用与推理延迟；对策包括本地缓存、内置数据兜底、超时降级与切换传统算法。综上，项目在既定资源约束下可完成开发与验证。

2.3 业务流程分析

系统的核心业务流程围绕用户访问、内容分发和行为反馈展开。用户进入系统后首先进行注册或登录，登录成功后进入首页。首页包含推荐流和关注流两种信息流类型。推荐流由系统根据用户画像和候选内容进行个性化排序，关注流则展示用户关注对象发布的内容。用户在浏览过程中产生点赞、点踩、评论、转发、搜索、跳过等行为，这些行为被写入行为表，并在后续画像构建和推荐排序中发挥作用。

图 2-1 系统总体业务流程图：用户登录后进入首页，系统构建候选池并生成推荐信息流，用户互动行为被记录后反哺用户画像和后续排序。

内容发布流程包括输入文本、选择图片、上传图片、调用发帖接口、保存内容、解析话题标签和异步 AI 打标等步骤。若用户输入正文中包含显式话题标签，系统会将标签保存到内容标签关联表；若没有显式标签，系统也可以通过 AI 打标服务生成语义标签。这样既满足用户主动标注需求，也能提高推荐算法可用的内容特征数量。

推荐信息流流程包括候选池构建、负反馈过滤、用户画像读取、协同过滤召回、多因子评分、作者多样性惩罚和加权随机采样。候选池包含网内来源和网外来源，网内来源来自用户关注对象发布的内容，网外来源来自全站公开内容。系统先过滤用户明确屏蔽或不感兴趣的内容，再对候选内容计算综合推荐分，最后通过加权随机采样生成具有一定差异性的结果列表。

兰州理工大学毕业设计

登录与鉴权子流程：用户提交凭证→服务端校验→签发 JWT→前端持久化→后续请求携带头。发帖子流程：可选上传→发布→标签解析→异步打标。广告子流程：读取配置→算分→插入列表→回写展示点击。通知子流程：互动事件→写通知表→WS 推送→前端更新未读。

异常与分支说明：登录失败包括账号不存在、密码错误、账号封禁三类，应在前端区分提示文案以降低用户枚举风险。发帖失败可能源于上传超时或正文为空，应先校验前端表单再调用接口。推荐流在候选为空时返回空数组并提示用户关注更多作者或等待数据导入。广告插入若 interval 大于列表长度则本轮可能无广告位，属正常边界。通知推送若 WebSocket 未连接，仍依赖通知列表接口补偿；私信同理。

2.4 数据流与数据字典

系统数据主要在前端页面、后端服务、数据库、AI 服务和外部数据源之间流动。前端通过 Axios 调用后端接口，通过 WebSocket 接收通知和私信；后端通过 Repository 层读写 MySQL 数据库，通过 AI 服务调用 Ollama 或 DeepSeek，通过 Kaggle 数据导入服务（KaggleImportService）接入公开数据集，并在外部数据源不可用或数据不足时使用内置内容库补齐。

图 2-2 系统一层数据流图：用户与管理员通过 Vue 前端访问后端 API，后端读写业务数据库，并调用推荐服务、AI 服务、WebSocket 服务和外部数据导入服务。

表 2-2 核心数据字典

数据项	含义	来源	去向
userId	用户唯一标识	登录用户、请求参数	内容、行为、关注、通知、推荐服务
contentId	帖子唯一标识	内容表	行为记录、评论、通知、推荐候选池
behaviorType	行为类型，如 VIEW、LIKE、COMMENT、REPOST、SEARCH	用户操作	用户画像、推荐算法、统计分析
tagName	标签名称	发帖解析、AI 打标	搜索、热门话题、标签匹配推荐
scoreBreakdown	推荐得分详情	推荐算法	前端评分面板、算法对比页面

兰州理工大学毕业设计

negativeSignal	负反馈信号	不感兴趣、屏蔽、静音操作	推荐过滤和降权处理
adScore	广告匹配分	广告服务	信息流广告排序与插入

补充数据项：negative_signal 记录（user_id、target_type、target_id、signal_type）支撑候选过滤；私信消息行（sender_id、recipient_id、content、is_read）支撑会话与未读统计；广告配置中的插入间隔 interval 决定前端合并列表的下标规则。上述项与表 2-2 主干字段共同构成端到端数据闭环。在二层分解中，可将推荐相关读路径抽象为相对独立的子过程（读取行为与内容、输出评分明细），与以事务写入为主的业务路径在职责上分离，便于后续按负载分别伸缩或部署。若写入峰值较高且需减轻画像更新对主库的压力，可演进为“先持久化行为事件、由消费者异步聚合刷新画像”的流水线（常配合消息队列或变更日志）；本课题实现仍以请求路径内的同步读写为主，该模式作为规模化场景下的可选架构方向。分层数据流同时标明了各子系统的数据输入输出与依赖关系，可作为扩展时划分服务边界与数据归属的参照。

图 2-3 二层数据流分解（逻辑示意）：前端请求经 API 网关至业务服务；业务服务写入数据库并触发推荐子过程；推荐子过程读取行为与画像返回排序列表。

2.5 非功能需求分析

性能与可扩展性：中小规模部署下单机即可满足需求；后续可通过 Redis 缓存热门话题与画像摘要、异步线程池执行 AI 打标降低接口尾延迟。安全性：当前实现侧重功能链路完整性，生产环境需加固密码存储、HTTPS、CSRF 与接口鉴权。可维护性：Java 包结构与 Vue 目录结构遵循约定，便于团队协作。可观测性：关键路径打印耗时日志可辅助定位推荐瓶颈。兼容性：前端采用现代浏览器特性，WebSocket 降级由 SockJS 处理。

2.6 需求与实现对应关系

本节将课题任务书中的条目与系统实现模块进行映射。（一）架构与基础平台：任务书要求 B/S、前后端分离、Spring Boot、Vue 与 MySQL，本系统在一套仓库内分别维护 backend 与 frontend 工程并完成联调；注册登录与角色区分通过 JWT 与角色字段（role）

兰州理工大学毕业设计

实现；帖文发布、浏览、多级评论、点赞、转发、引用对应内容实体（Content）及其 REST 接口；关注关系对应关注实体（Follow）与关注流查询；关键词与 Hashtag 搜索由搜索控制器（SearchController）与正文标签解析协同完成；热门话题由标签统计与互动加权模块生成；基于 WebSocket 的私信与基于通知表的站内提醒满足实时通信任务描述。

（二）行为采集与画像：行为记录表（Behavior）统一承载多维事件；画像模块输出兴趣标签、活跃时段、用户类型与行为强度；负反馈实体用于屏蔽作者或隐藏帖子并在候选阶段剔除。（三）信息流排序：混合多因子打分涵盖互动加权、互动率、网内加成、标签匹配、TF-IDF、分段衰减、热门话题、作者多样性、加权随机采样与协同过滤；管理员可在传统混合推荐策略（Hybrid）与 AI 推荐策略（AiRecommendationStrategy）之间切换。（四）AI 与广告：对话式助手调用 DeepSeek API；信息流广告按间隔插入并按标签、出价与 CTR 排序。（五）可视化实验：算法对比页面（CompareView）集成对比指标、漏斗图与权重调节组件（WeightTuner）。（六）数据与测试：支持种子数据、Kaggle 数据集导入与内置内容库补齐；第 4 章给出用例表与结果分析。

第 3 章 系统总体设计

3.1 系统总体架构设计

系统采用 B/S 架构和前后端分离模式。浏览器端运行 Vue 3 单页应用，负责页面展示、用户交互、状态维护和图表渲染；后端采用 Spring Boot 提供 RESTful API 和 WebSocket 服务；数据库采用 MySQL 存储用户、内容、行为、标签、关注、通知、私信、广告等业务数据；AI 服务通过 Ollama 和 DeepSeek API 提供内容打标、AI 推荐和智能问答能力；外部数据导入模块用于导入 Kaggle 公开数据集，并在导入不足时使用内置内容库补齐候选内容。

前后端分离使界面迭代与算法迭代可并行：前端专注交互与可视化，后端专注业务一致性与数据完整性。推荐链路在逻辑上可分为数据层（行为与内容持久化）、特征层（画像与 TF-IDF 向量）、排序层（Hybrid 或 AI 策略）、展示层（信息流混合广告）。异常与降级集中在排序层实现，避免向上层泄漏实现细节。

图 3-1 系统总体架构图：浏览器访问 Vue 前端，前端通过 Axios 和 WebSocket 与 Spring Boot 后端通

兰州理工大学毕业设计

信，后端连接 MySQL、AI 服务和外部数据源。

后端分层设计能够提高系统可维护性。controller 层负责请求入口，service 层负责业务规则和算法逻辑，repository 层负责数据访问，model 层定义 JPA 实体，dto 层封装前后端交互对象，config 层管理跨域、安全、WebSocket 和文件映射等配置。推荐系统相关逻辑集中在 service 层，便于传统推荐策略、AI 推荐策略和策略管理器之间解耦。DTO 中的评分拆分对象（ScoreBreakdown）、画像响应对象（PersonaResponse）等与前端字段一一对应，减少重复拼装字段。

3.2 功能模块设计

系统功能模块按照业务领域划分为用户认证与资料模块、内容发布与互动模块、社交关系模块、行为采集模块、推荐排序模块、用户画像模块、搜索与热门话题模块、通知与私信模块、广告分发模块、AI 智能模块、管理后台模块和数据导入模块。各模块之间通过 REST API、WebSocket 推送与共享数据库表协作：行为采集模块为画像与推荐提供输入；推荐排序模块依赖画像输出动态权重；通知与私信模块依赖内容与行为事件触发；广告分发模块依赖画像标签与广告定向标签匹配。

图 3-2 系统功能模块结构图：系统由用户端业务模块、推荐算法模块、AI 智能模块、广告模块、管理后台和数据导入模块共同组成。

从软件工程角度，每个核心模块可归纳为输入、处理逻辑、输出与异常处理四个要素，以保证后续实现与测试有据可依。用户认证与资料模块的输入为注册信息、登录凭证与资料编辑表单；处理包括用户名唯一性校验、密码校验、封禁状态检查、JWT 生成与返回；输出为用户实体与 Token；异常包括用户名冲突、密码错误、账号封禁等。内容发布与互动模块的输入为帖子正文、图片、评论父节点及转发引用关系；处理包括上传文件映射、内容持久化、标签解析与异步 AI 打标；输出为 Content 实体及互动计数更新；异常包括上传失败、内容为空、删除权限不足等。

社交关系模块维护关注者标识（followerId）和被关注者标识（followeeId），输入为关注或取关请求；处理包括重复关注过滤、双向列表查询；输出为关注状态与列表数据；异常包括关注自身、目标用户不存在等。行为采集模块输入为前端上报的行为类型与可选停留时长；处理为写入 behaviors 表并在部分场景更新内容计数；输出为行为记录 ID；

兰州理工大学毕业设计

异常包括未登录、非法类型等。推荐排序模块输入为用户 ID 与分页参数；处理为候选池构建、策略选择、打分与采样；输出为排序后的帖子列表及可选评分拆分对象（ScoreBreakdown）；异常包括候选为空、AI 服务超时触发降级等。

用户画像模块输入为用户 ID 与时间窗口；处理为聚合行为、统计标签与偏好并计算动态权重集合（DynamicWeights）；输出为画像 DTO 供推荐与广告使用；异常包括冷启动用户数据不足时返回默认权重。搜索与热门话题模块输入为关键词与分页；处理为全文或字段检索、话题聚合；输出为帖子、用户、话题等多类型结果；异常包括关键词过短返回提示。通知与私信模块输入为互动事件或私信载荷；处理为持久化并经由 WebSocket 推送；输出为通知列表或未读数；异常包括接收者离线仅保留数据库记录。广告分发模块输入为用户画像与广告配置；处理为匹配分计算与间隔插入；输出为广告卡片与统计字段；异常包括无可用广告时跳过插入。

表 3-1 核心模块输入输出概要

模块	主要输入	主要输出	典型异常
用户认证与资料	用户名、密码、资料字段	用户信息与 Token	用户名重复、账号封禁
内容与互动	帖子、评论、转发参数	Content 与计数更新	上传失败、权限不足
行为采集	行为类型、contentId、时长	Behavior 记录	类型非法、未登录
推荐排序	userId、分页	排序 Feed、评分明细	候选为空、AI 降级
用户画像	userId	标签分布、动态权重	冷启动默认策略
通知与私信	事件、消息体	通知记录、WS 推送	连接断开仅落库
广告分发	画像标签、广告配置	排序广告、展示统计	无启用广告

3.3 数据库设计

数据库设计围绕用户、内容、行为、社交关系和推荐辅助数据展开。核心实体包括用户、帖子、行为、标签、关注、通知、私信、负反馈、广告和广告配置。用户可以发

兰州理工大学毕业设计

布多条内容，可以产生多条行为记录，也可以关注多个用户；内容可以拥有多个标签，可以触发多条通知，也可以作为评论、转发和引用的目标；广告根据标签、类别、出价和历史效果参与信息流插入。

图 3-3 系统 E-R 图：用户与内容、行为、关注、通知、私信之间形成一对多关系，内容与标签之间形成多对多关系，广告与广告配置共同支持投放。

表 3-2 用户表设计

字段	类型	说明
id	BIGINT	用户 ID
username	VARCHAR	用户名，唯一
handle	VARCHAR	用户昵称标识，如 @user1
avatarUrl	VARCHAR	头像地址
bio	VARCHAR	个人简介
password	VARCHAR	登录密码
role	VARCHAR	USER 或 ADMIN
banned	BOOLEAN	是否封禁
customWeights	TEXT	用户自定义推荐权重 JSON
createdAt	DATETIME	创建时间

表 3-3 内容表设计

字段	类型	说明
id	BIGINT	帖子 ID
author	BIGINT	作者 ID
parentContent	BIGINT	父评论 ID
content	TEXT	正文内容
imageUrl	VARCHAR	图片地址
category	VARCHAR	内容分类
viewCount	INT	浏览数
likeCount	INT	点赞数
commentCount	INT	评论数
dislikeCount	INT	点踩数
repostCount	INT	转发或引用数

兰州理工大学毕业设计

createdAt	DATETIME	发布时间
-----------	----------	------

表 3-4 行为表设计

字段	类型	说明
id	BIGINT	行为 ID
userId	BIGINT	用户 ID
contentId	BIGINT	内容 ID，搜索等行为可为空
type	VARCHAR	VIEW、LIKE、DISLIKE、SKIP、SEARCH 等
duration	INT	浏览停留时长
createdAt	DATETIME	行为发生时间

关注表用于表达用户之间的有向社交关系，是“网内候选”的数据来源：若 followerId 关注 followeeId，则 followeeId 发布的内容可优先进入该 followerId 的候选池并给予 In-Network 加成。标签系统通常采用标签实体表与内容标签关联表的多对多结构：标签名称唯一，内容与标签通过中间表关联，便于热门话题统计与检索索引。通知表记录点赞、评论、关注等事件型消息；私信表或消息实体记录会话与聊天内容，可与 WebSocket 推送配合实现实时性。

表 3-5 关注表设计概要

字段	类型	说明
id	BIGINT	主键
followerId	BIGINT	关注者用户 ID，逻辑外键指向 users.id
followeeId	BIGINT	被关注者用户 ID
createdAt	DATETIME	关注时间，可用于推荐中新关注加权

表 3-6 标签实体与内容标签关联（逻辑结构）

对象	说明
tags	标签表：id、name（唯一）、可选统计字段。
content_tags	关联表：contentId、tagId，联合唯一约束防止重复绑定。

兰州理工大学毕业设计

表 3-7 通知表设计概要

字段	类型	说明
id	BIGINT	主键
recipientId	BIGINT	接收者
actorId	BIGINT	触发者
type	VARCHAR	LIKE、COMMENT、 FOLLOW、REPOST、QUOTE 等
entityId	BIGINT	关联帖子或内容 ID
isRead	BOOLEAN	是否已读
createdAt	DATETIME	通知时间

表 3-8 广告表设计概要

字段	类型	说明
id	BIGINT	主键
title、 description、imageUrl	VARCHAR/TEXT	展示文案与素材
targetUrl	VARCHAR	点击跳转地址
advertiser	VARCHAR	广告主标识
targetTags、 category	VARCHAR	定向标签与类别
bidPrice	DOUBLE	出价，用于排序加权
impressionCount 、clickCount	INT	展示与点击计数
active	BOOLEAN	是否启用投放

表 3-9 负反馈信号表 (negative_signals)

字段	类型	说明
id	BIGINT	主键，自增
user_id	BIGINT	发起负反馈的用户
target_type	枚举	CONTENT（单条内容）或 AUTHOR（作者）
target_id	BIGINT	目标内容 ID 或用户 ID

兰州理工大学毕业设计

signal_type	枚举	NOT_INTERESTED、BLOCK、MUTE
created_at	DATETIME	创建时间

表 3-10 私信消息表 (tb_message)

字段	类型	说明
id	BIGINT	主键
sender_id	BIGINT	发送方用户 ID
recipient_id	BIGINT	接收方用户 ID
content	TEXT	消息正文
created_at	DATETIME	发送时间
is_read	BOOLEAN	是否已读

负反馈表与推荐候选过滤直接对应：target_type=AUTHOR 时在候选阶段剔除该作者全部帖子；target_type=CONTENT 时剔除指定帖子。signal_type 区分不感兴趣、屏蔽与静音的业务语义，便于后续扩展撤销策略。私信表采用扁平消息行存储会话消息；会话列表可由双方 userId 聚合查询实现；本文未单独引入会话头表以降低建模复杂度。删除用户时的级联策略应在生产环境通过外键或定时清理任务补齐。

数据库设计的冗余与一致性方面：内容表的浏览数（viewCount）、点赞数（likeCount）等计数字段属于适度冗余，可通过事务或异步修正保持一致性，换取列表页与推荐打分时的读取性能。行为表保留停留时长字段（duration）支撑停留时长画像；搜索行为（SEARCH）的内容标识（contentId）可为空，以表达“主动检索意图”而不绑定单一帖子。负反馈相关实体用于屏蔽作者、隐藏帖子等，推荐流水线在候选合并后立即过滤，避免无效打分消耗。

3.4 推荐算法设计

推荐排序模块采用策略模式设计，统一定义推荐策略接口（RecommendationStrategy），传统混合推荐策略（HybridRecommendationStrategy）和 AI 推荐策略（AiRecommendationStrategy）均实现该接口。推荐策略管理器

兰州理工大学毕业设计

(RecommendationStrategyManager) 持有两种策略实例并维护全局策略标识 (如 traditional、ai) , 推荐服务 (RecommendationService) 在组装候选池后通过 getActiveStrategy() 获取当前策略执行排序, 从而避免在业务代码中硬编码单一算法实现。管理员通过 PUT /api/admin/rec-strategy 切换策略后, 全体用户的 For You 请求立即生效, 从而在线对比两种策略的输出差异。该模式符合开闭原则: 新增策略仅需实现接口并注册到管理器即可, 也便于在个性化重排序实验中扩展新的策略分支[5][12]。

(1) 候选池与双源机制。参考公开信息流架构中的多源候选思想[15], 本系统将候选拆分为 In-Network 与 Out-of-Network。In-Network 来自当前用户所关注用户发布的最新内容, 体现社交关系的信任传递与兴趣同质性; Out-of-Network 来自全站公开池, 用于探索用户尚未关注但可能感兴趣的作者与话题。两路候选合并去重后形成统一列表, 再进入过滤与打分。候选规模可通过上限截断与时间窗口控制, 避免单次请求遍历全库。

(2) 负反馈与合规过滤。负反馈信号包括屏蔽作者、对单条内容不感兴趣、静音等, 存入独立实体后在候选阶段直接剔除或强降权, 减少无效曝光与用户挫败感。该步骤位于打分之前, 可降低后续 TF-IDF、协同过滤等计算开销。

(3) 多行为加权互动分。不同行为类型对用户兴趣的揭示强度不同, 一般转发、评论高于点赞, 点赞高于浏览。系统对用户历史行为赋权并叠加时间衰减, 形成 engagement 分量; 互动率因子可采用点赞加评论量与浏览量的比值刻画内容“有效密度”, 抑制刷量低质内容。

(4) TF-IDF 与余弦相似度。对文本内容分词后, 词项 i 在文档 d 中的 TF-IDF 权重采用式 (3-1) 计算, 其中 N 为语料文档数, df_i 为包含词 i 的文档数, $tf_{i,d}$ 为词项 i 在文档 d 中的词频。

$$w_{\{i,d\}} = tf_{\{i,d\}} \times \log(N / df_i) \quad (3-1)$$

系统在用户画像侧使用近期互动内容构造兴趣向量, 在候选帖子侧构造内容向量, 并按式 (3-2) 计算用户 u 与候选内容 c 的余弦相似度。该相似度作为内容相似度加成 (tfidfSimilarityBoost) 的主要来源, 用于刻画语义层面的兴趣匹配, 从而弥补纯标签匹配粒度较粗的问题。

$$\text{sim}(u,c) = \sum_i w_{\{i,u\}} w_{\{i,c\}} / (\|u\| \cdot \|c\|) \quad (3-2)$$

兰州理工大学毕业设计

用户画像构建与行为偏好抽取的相关研究也说明，文本内容、浏览行为和显隐式反馈可以共同服务于个性化推荐[7][8]。

(5) 协同过滤加成。协同过滤服务 (CollaborativeFilteringService) 基于用户—物品交互矩阵，结合多行为权重与时间衰减计算用户相似度，产出 Top-K 相似用户及其偏好内容集合；若候选帖子 ID 落在协同推荐集合中，则给予协同过滤加成 (collaborativeBoost)，并可由动态权重集合 (DynamicWeights) 中的协同权重进行调节。该方法能够在标签稀疏或新话题出现时利用群体智慧补全兴趣，与图卷积协同过滤、矩阵分解和基于物品协同过滤等推荐思想具有一致性[2][16][17]。

(6) 时间衰减分段函数。设帖子年龄为 ageHours，可对不同区间施加不同衰减速率：黄金期 (0–6h) 衰减慢以突出时效；常规期 (6–24h) 线性或对数衰减；长尾期 (72h+) 加速衰减但仍保留非零底值以保留优质存量内容曝光机会。最终 timeDecay 作为除数作用于 engagement，实现“越新越靠前，但不绝对压制长尾”。

(7) 作者多样性与探索。对同一作者 (author) 在候选列表中的多条内容施加递减系数，避免单一来源刷屏；随机扰动项 (jitter) 或加权随机洗牌在高分区间内引入可控随机性，实现探索与利用折中，缓解过滤气泡。评分拆分对象 (ScoreBreakdown) 将各分项写入 DTO，前端可展开查看，以支撑推荐结果的可解释性。该思路与推荐系统中多样性、质量偏好和去偏建模的研究方向相呼应[20]。

多因子综合评分可概括为：

(8) 综合评分。传统混合推荐策略在候选过滤后计算最终推荐得分 (finalScore)。为避免公式过长，式 (3-3) 采用缩写表示各分项，其中 E 为互动得分 (engagement)，D_t 为时间衰减因子 (timeDecay)；B_topic、B_author、B_personal、B_tfidf、B_trend、B_cf、B_depth、B_fresh 分别表示话题亲和度、作者亲和度、个性化加成、内容相似度加成、热门话题加成、协同过滤加成、内容深度匹配加成和新鲜度偏好加成，ε 为随机扰动项 (jitter)。

$$\text{finalScore} = E / D_t + B_topic + B_author + B_personal + B_tfidf + B_trend + B_cf + B_depth + B_fresh + \varepsilon \quad (3-3)$$

其中互动得分 (engagement)、话题亲和度 (topicAffinity)、协同过滤加成 (collaborativeBoost) 等分项均由用户行为画像服务 (UserBehaviorProfileService) 输出的

兰州理工大学毕业设计

动态权重集合（DynamicWeights）进行加权组合，使冷启动用户自动提高热门与探索权重，活跃用户自动提高个性化与协同权重。

（8）重复曝光惩罚。重复曝光惩罚在最终推荐得分（finalScore）计算完成后执行，用于降低用户已明确反馈内容的再次曝光概率。与项目代码中的常量保持一致：已点踩内容乘以点踩惩罚因子（DISLIKE_PENALTY_FACTOR，取值 0.02），已点赞内容乘以点赞惩罚因子（LIKE_PENALTY_FACTOR，取值 0.15），已浏览内容乘以浏览惩罚因子（VIEWED_PENALTY_FACTOR，取值 0.5）。其中点踩属于强负反馈，惩罚最重；已点赞和已浏览内容并非完全屏蔽，而是降低优先级，以便在保留少量回访可能的同时提升新内容曝光。

（9）AI 推荐与降级。AI 推荐策略（AiRecommendationStrategy）将画像摘要与候选摘要（含 id、正文片段、标签、互动统计）封装为提示词（Prompt），约束模型仅输出 JSON 结构，其中排序字段（ranking）表示候选顺序，理由字段（reasons）表示推荐解释。解析成功则重排列表；若 Ollama 不可用、超时或 JSON 非法，则降级为按基础互动分排序并在前端标识降级模式，保障系统可用性。该设计并非替代传统推荐模型，而是在混合排序基础上增加智能解释与对比实验入口，与深度学习推荐系统中“特征理解 + 排序优化”的发展方向保持一致[11][14][18][19]。

（10）加权随机采样。对候选按最终推荐得分（finalScore）降序排序后，若直接截取 Top-N，则每次刷新结果高度相似，列表多样性不足。工程上可在前 M 条候选中按式（3-4）计算前缀和并抽样：先计算前缀和 S_k，再取随机数 r 进行二分定位，重复直至凑满页大小。该机制对应课题要求中的加权随机采样与探索—利用平衡，使低分但非零分内容仍具曝光机会。若需可复现实验，可固定随机种子或关闭采样仅用于对比基线。

$$S_k = \sum_{i=1}^k w_i, r \in (0, S_M) \quad (3-4)$$

（11）作者多样性惩罚的窗口化描述。在按分数初排后，可沿列表滑动长度为 L 的窗口，统计同一 author 出现次数；当次数超过阈值 t 时，对后续同作者候选追加惩罚项或将其后移。该策略与“同一作者刷屏”问题直接对应，且与第（10）条采样可组合：先窗口重排再加权抽样，或先抽样再轻量去重。隐式反馈（浏览、停留）在协同侧可视为弱正信号，与显式反馈联合建模的思想与融合显隐式反馈的推荐研究一致[4][16]。

3.5 用户画像设计

用户画像模块由行为数据驱动生成。系统读取用户历史行为记录后，查询相关内容和标签，并根据行为类型赋予不同权重。一般而言，转发和评论代表较强兴趣，点赞代表正向兴趣，浏览代表弱兴趣，点踩、跳过和屏蔽代表负向兴趣。系统还结合行为发生时间进行衰减，使近期行为对画像影响更大。搜索行为（SEARCH）记录用户主动检索意图，可与标签偏好共同用于查询类兴趣发现；浏览行为（VIEW）附带的停留时长用于推断属于“浅阅读”还是“深度阅读”，进而调节内容深度匹配加成（depthMatchBoost）与新鲜度偏好加成（freshnessBoost）的权重。

图 3-4 用户画像构建流程图：读取行为记录，按行为类型和时间赋权，统计兴趣标签、作者偏好、互动风格、内容深度偏好和新鲜度偏好，最终生成动态推荐权重。

画像输出通常包括：兴趣标签分布（标签名与权重）、作者偏好列表、活跃时段直方图、用户类型（创作者、互动型、浏览型）、行为强度统计、内容长度偏好与新鲜度偏好。动态权重集合（DynamicWeights）将这些偏好映射为推荐公式中各分项的系数，例如提高协同过滤权重（wCollaborative）、文本相似度权重（wTfidf）或热门话题权重（wTrending）。冷启动阶段若行为稀少，则回退到全局热门、时间新鲜度与随机探索，并逐步过渡到个性化；该策略与用户画像构建和深度学习推荐系统中对用户兴趣表征的讨论一致[8][11]。

系统将用户阶段划分为冷启动用户、初级用户和活跃用户。冷启动用户行为较少，推荐策略更依赖热门内容、时间新鲜度和随机探索；初级用户已有一定行为数据，推荐策略采用热门与个性化混合；活跃用户行为数据较丰富，推荐策略更依赖兴趣画像、协同过滤和作者偏好。通过不同阶段的动态权重调整，系统能够适应用户行为数据从少到多的变化过程。

表 3-11 用户阶段与推荐策略

用户阶段	判定条件	推荐策略特点
冷启动用户	行为数少于 10	更多依赖热门内容、新鲜内容和探索因子。
初级用户	行为数 10 到 49	热门内容与个性化标签匹配混合排序。
活跃用户	行为数不少于 50	更多依赖个性化画像、协同过

3.6 安全与实时通信设计

系统登录成功后由后端生成 JWT 令牌（JWT Token）并返回给前端，前端将用户信息与令牌持久化于 localStorage，并在 Axios 全局拦截器中附加 Authorization 头。当前实现中，Spring Security 采用无状态配置，JWT 认证过滤器（JwtAuthFilter）主要完成令牌解析与上下文注入，接口层以功能联调和演示为重点；前端路由与管理页面根据管理员角色标识（role=ADMIN）控制入口，部分敏感操作仍需要在服务端进一步补充强制权限校验。正式部署时应将全放行策略（permitAll）调整为按角色授权，并补充 BCrypt 密码哈希、刷新令牌、接口审计日志和敏感操作拦截。

实时通信模块基于 WebSocket、SockJS 与 STOMP 协议实现可靠传输与浏览器兼容。后端配置消息代理与用户目的地前缀，使通知推送到 /user/{userId}/queue/notifications 等队列；前端订阅后在 UI 上增量更新未读徽标。私信同理可走独立队列或 REST 轮询兜底。断线重连时，数据库中的通知与消息记录仍保留，用户刷新页面可补偿展示，满足“实时优先、最终一致”的工程目标。

安全细节方面，生产环境应为 WebSocket 启用同源策略与令牌校验，防止未订阅他人队列；当前实现侧重功能链路验证，若暴露于公网必须补齐握手阶段的身份绑定。

总结而言，认证授权解决“谁能调用接口”，实时通信解决“如何把事件送达在线会话”，二者共同构成社交应用的体验底座。

后续若引入多设备同时在线，可在用户维度之外增加设备维度的推送策略，并与离线推送互为补充，形成连续可用的通知链路。

3.7 广告分发设计

信息流广告与社交内容共享同一浏览上下文，需兼顾用户体验与商业目标。系统在服务端或前端插入环节读取广告配置（如每隔 N 条帖子插入一条广告），再调用广告服务按用户画像兴趣标签与广告定向标签（targetTags）计算广告匹配分（adScore），并结

兰州理工大学毕业设计

合出价（bidPrice）与历史点击率（CTR）得到质量分。排序最高的若干条广告进入展示队列；展示与点击通过接口回写展示次数（impressionCount）和点击次数（clickCount），形成闭环优化数据。

广告与推荐共享画像数据源，但优化目标不同：推荐侧重长期留存与点击多样性，广告侧重转化与收益，因此二者公式分离且可通过配置限制每日曝光频次（可在后续版本扩展）。当前设计对应课题需求中的“标签匹配 + 出价加权”投放思路。

从指标体系看，展示次数反映库存消耗，点击次数反映创意吸引力，点击率（CTR）则是二者的比值；若仅优化标签匹配而忽略点击率，可能出现“匹配但不点击”的低效投放。因此质量分（qualityScore）引入历史点击率作为校正项，能够在新广告缺少统计时退化为中性权重，随着样本累积逐步提高区分度。若未来接入转化率埋点，还可进一步扩展为后置 ROI 模型。

3.8 算法可视化设计

算法验证页（CompareView）承担对比评测与结果解释职能：其一，双栏对比个性化排序与时间倒序，输出平均分、标签命中率、TF-IDF 均值、互动量与个性化倍率等统计量；其二，多用户画像对比展示不同用户的动态权重与 Top 推荐，体现“千人千面”；其三，排序管道漏斗图呈现候选规模在各过滤阶段的变化；其四，权重调节组件（WeightTuner）支持在线调节权重，并通过 `/api/compare/tuned` 重新计算推荐结果，验证参数敏感性。图表采用 ECharts，词云与雷达图用于画像页，以支撑数据驱动的观察与记录。

可视化模块强调先呈现中间结果再给出结论：读者可通过漏斗图观察候选在各阶段的收缩过程，通过双栏指标比较个性化排序相对时间序的差异，通过权重调节组件（WeightTuner）观察权重调整对指标的影响方向。

3.9 核心 REST 接口与 DTO 约定

为保证前后端并行开发与后期测试可追溯性，系统在控制器层采用统一 REST 风格资源命名，DTO 承载结构化输出，避免直接泄露实体内部字段。认证接口返回认证响应对象（AuthResponse），内含令牌（token）与用户概要；内容接口返回内容传输对象

兰州理工大学毕业设计

（ContentDTO）列表或分页包装；推荐接口支持附带评分拆分对象（ScoreBreakdown）的扩展字段以便调试；管理员接口统一前缀 /api/admin。错误响应包含 HTTP 状态码与简要消息（message），前端 Axios 拦截器集中提示。

表 3-12 主要 REST 接口一览（节选）

路径	方法	说明
/api/auth/register	POST	注册并签发 Token
/api/auth/login	POST	登录校验与签发 Token
/api/content/feed	GET	个性化或时间序信息流
/api/content/publication	POST	发帖
/api/behavior/view	POST	浏览行为与停留时长
/api/search	GET	综合搜索，附带 userId 记录 SEARCH
/api/compare/feed	GET	算法对比数据
/api/compare/tuned	GET	调参后的对比数据
/api/admin/recommendation-strategy	GET/PUT	查询或切换推荐策略
/api/user/{id}/persona	GET	用户画像

DTO 命名遵循 Java Bean 规范，前端按接口字段约定进行渲染与状态维护，可减少字段遗漏和命名不一致。对于大数据量导入与 AI 推理等耗时操作，可采用异步任务 ID 查询模式扩展；当前版本仍以同步返回和缓存预计算为主，以保持实现简洁。

3.10 关键术语与符号说明

为便于阅读，对文中频繁出现的术语、符号与缩写集中说明。下列定义侧重于本文语境下的含义，不等同于某一文献中的唯一标准定义。

表 3-13 术语与缩写

术语或符号	含义说明
In-Network	网内候选，主体来自当前用户关注对象发布的内容。
Out-of-Network	网外候选，主体来自全站公开内容，用于兴趣探索。

兰州理工大学毕业设计

TF-IDF	词频—逆文档频率，用于构建文本向量并与兴趣向量计算相似度。
CTR	点击率，click/impression，用于广告质量评估。
JWT	JSON Web Token，一种紧凑的令牌格式，用于无状态会话。
REST	表述性状态转移风格的 HTTP API 设计范式。
STOMP	面向消息的文本协议，常与 WebSocket 组合实现订阅推送。
DTO	数据传输对象，封装接口响应字段。
ScoreBreakdown	推荐得分拆分对象，记录各分项贡献。
DynamicWeights	动态权重集合，由画像阶段输出并作用于打分公式。
Ollama	本地大模型运行环境，本课题用于打标与 AI 推荐。
冷启动	新用户或新内容缺少行为历史时的推荐难题。
探索—利用	Exploration-Exploitation，在尝试新内容与巩固已知兴趣之间权衡。

表 3-14 主要数学符号（与正文约定一致）

符号	含义
u, c	分别表示用户与候选内容。
$w_{i,d}$	词项 i 在文档 d 上的 TF-IDF 权重。
$\text{sim}(u, c)$	用户兴趣向量与候选内容向量的余弦相似度。
ageHours	内容自发布时刻起的小时年龄，用于时间衰减。
finalScore	推荐最终得分，由多分项线性或分段组合得到。

上述术语表随系统迭代可能新增字段（例如新增行为类型或广告约束），应以实现中的枚举与配置为准保持一致。

3.11 推荐流水线与复杂度分析

以下用文字化步骤描述一次 For You 请求的推荐流水线，将第 3、4 章涉及的接口与类组织为连贯的处理序列，并对单机环境下的时间复杂度做数量级估计。

步骤一：身份与会话。前端附带 JWT，后端过滤器解析 `userId` 并注入上下文；当前演示版本以软校验为主，前端在本地会话缺失时跳转登录，后续可扩展为服务端强制 401 拦截。该步骤时间复杂度 $O(1)$ 。

兰州理工大学毕业设计

步骤二：候选收集。查询关注用户近期发帖构成网内集合 S_{in} ，查询全站候选构成网外集合 S_{out} ；合并为候选集合 C ，并按内容标识（`contentId`）去重。若采用上限截断，复杂度约为 $O(K_{in} + K_{out})$ ，其中 K 为截断规模。

步骤三：负反馈过滤。读取屏蔽列表与不感兴趣集合，从 C 中剔除；若使用哈希集合判断，总体近似 $O(|C|)$ 。

步骤四：画像与协同。读取用户画像向量与协同过滤 **Top** 列表；画像聚合复杂度与用户历史行为条数 B 成正比，约为 $O(B)$ ；协同过滤若预先缓存相似用户结果则可降为 $O(1)$ 查询。

步骤五：逐条打分。对候选集合 C 中每条内容计算评分拆分对象（**ScoreBreakdown**）；若 **TF-IDF** 需要向量点积，长度取决于词典维度 L ，则单条 $O(L)$ ，整体 $O(|C| \times L)$ 。在本文数据规模下该路径可接受；若迁移到大规模生产，应改为离线特征缓存或近似最近邻检索。

步骤六：排序与多样性。按分数排序 $O(|C| \log |C|)$ ；作者多样性惩罚可能采用滑动窗口或频次哈希，近似线性。

步骤七：加权随机洗牌。为列表引入可控抖动，避免每次刷新完全一致；实现可使用前缀和与二分或权重蓄水池抽样。

步骤八：策略分支。若启用 **AI** 策略，则将候选摘要拼接 **Prompt** 调用 **Ollama**；主导耗时来自网络推理，可达数秒；若超时则降级为步骤五结果按互动分重排。

步骤九：分页返回。截取当前页返回前端并附带可选评分明细字段。

综上，传统路径瓶颈主要在逐条 **TF-IDF** 与排序；**AI** 路径瓶颈在外部推理延迟。后续可通过批量向量缓存、倒排索引与异步预热画像缓解。

第 4 章 系统详细设计及实现

本章从工程落地角度说明系统如何在既定架构下完成编码实现与验证。实现阶段遵循三项原则：其一，业务写入路径与推荐读取路径分离——发帖、点赞等行为先保证事务一致写入数据库，再由推荐服务异步或同步聚合画像；其二，算法策略与候选构建分离——候选池逻辑保持稳定，策略仅负责打分与重排，便于单元测试与策略替换；其三，交互可用性与健壮性兼顾——**AI** 相关调用必须具备超时与降级分支，避免单次推理阻塞

兰州理工大学毕业设计

整体信息流请求。

测试阶段遵循“场景驱动 + 指标对照”方法：场景驱动覆盖用户侧与管理侧完整故事线；指标对照依赖 CompareView 输出的平均分、标签命中率、TF-IDF 均值等量化差异，避免仅凭主观观感评价推荐优劣。对于表格中标记为通过的用例，均在本地开发环境重复执行至少两次以降低偶然性。

4.1 开发环境与项目结构

系统开发环境以 Windows 为主，后端运行环境为 JDK 17 和 Maven，前端运行环境为 Node.js 和 npm，数据库采用 MySQL，后端默认端口为 8888，前端默认端口为 5173。后端服务启动后提供 `http://localhost:8888/api` 作为 API 基础地址，WebSocket 地址为 `http://localhost:8888/ws`，前端通过浏览器访问 `http://localhost:5173`。

表 4-1 开发环境配置

项目	配置
操作系统	Windows
后端运行环境	JDK 17、Maven
前端运行环境	Node.js、npm
后端框架	Spring Boot 3.2.1
前端框架	Vue 3、Vite、Element Plus
数据库	MySQL rec_db
图表工具	ECharts、echarts-wordcloud
AI 服务	Ollama、DeepSeek API

项目目录分为 backend、frontend、scraper 与论文文档等部分。backend 下 `src/main/java/com/example/rec` 包含 controller（REST 入口）、service（业务与算法）、repository（JPA）、model（实体）、dto（传输对象）、config（全局配置）；`src/main/resources/application.properties` 配置数据源、JPA、跨域、文件上传路径、Ollama 模型名与温度等。frontend 下 `src/views` 为页面级路由组件（如 HomeView、AdminView、CompareView），`src/components` 为可复用组件（如信息流列表组件 FeedList、帖子卡片组

兰州理工大学毕业设计

件 TweetCard、权重调节组件 WeightTuner），src/router 定义路由守卫与懒加载。scraper 存放辅助脚本或抓取逻辑，与后端批量导入接口协同。

构建与运行命令：后端在 backend 目录执行 mvn spring-boot:run 启动服务，也可执行 mvn -q test 或打包命令进行编译与依赖装配校验；前端在 frontend 目录执行 npm install、npm run dev 本地调试、npm run build 生产构建。数据库需预先创建 rec_db 并导入初始化脚本或依赖 DataSeeder 自动生成种子数据。

前端路由侧，views 目录按业务分包：首页与个人主页承担消费路径；CompareView 专注算法对比展示；AdminView 聚合后台治理；AdDashboard 承担广告报表；GrokView 承载 AI 助手。通过路由懒加载减轻首屏体积，路由 meta 可挂载标题与权限标记，便于后续接入全局导航守卫统一鉴权。

4.2 部署配置与运维要点

本节面向单机部署与小型局域网访问场景，给出可复现的部署步骤与常见问题排查思路，以降低环境迁移成本。

（一）依赖安装：后端需 JDK 17、Maven 3.8+；前端需 Node.js 18+ 与 npm；数据库需 MySQL 8.x；AI 功能需本机安装 Ollama 并拉取打标模型与推荐模型镜像；DeepSeek API 需在配置文件中填写密钥并注意外网连通性。操作系统以 Windows 为例时，应注意防火墙放行 8888、5173 端口以便浏览器访问。

（二）数据库初始化：创建数据库 rec_db，配置字符集 utf8mb4 以兼容 emoji 与多语言正文；导入建表脚本或首次启动依赖 JPA ddl-auto 自动生成（开发环境可选用）。随后运行 DataSeeder 或导入 JSON，确保至少包含多领域标签帖子与多名模拟用户，以便协同过滤与热门话题模块有足够信号。

（三）配置文件：application.properties 中需核对数据源 URL、用户名密码、文件上传目录映射、跨域来源列表、Ollama 服务地址与模型名称（打标模型与推荐模型分离）、DeepSeek API Key、JWT 密钥长度与过期策略。修改配置后应重启后端进程使变更生效。

（四）启动顺序：建议先启动 MySQL，再启动 Ollama（若使用 AI），然后 mvn spring-boot:run 启动后端，最后 npm run dev 启动前端。若前端使用代理转发 API，应检查

兰州理工大学毕业设计

vite.config 中 proxy 目标端口与后端一致，否则将出现跨域或 404。

（五）功能巡检清单：登录注册成功后访问首页；切换 For You 与 Following；执行点赞、评论、搜索并观察行为是否入库；打开 CompareView 查看对比指标；进入 Admin 切换推荐策略并返回首页确认徽标；关闭 Ollama 再次请求 AI 模式验证降级。若某一步失败，应优先查看浏览器控制台网络面板与后端控制台堆栈。

（六）性能与日志。中小规模负载下通常无需 JVM 大规模调优；若 AI 推理耗时过长，可降低候选摘要条数或提升超时阈值并启用降级。建议在 ContentController 与 RecommendationService 的关键路径打印耗时日志，以便定位瓶颈。

（七）备份与版本管理。数据库定期导出 SQL；前端 package-lock.json 与后端 pom.xml 锁定依赖版本；Git 可使用发布标签标记稳定版本，避免在未完成特性合并后即部署验证。

4.3 用户认证与内容互动实现

认证链路。注册 POST /api/auth/register：服务端校验用户名唯一性，密码明文入库仅为开发测试阶段的简化处理，生产应替换为单向哈希。登录 POST /api/auth/login：根据用户名加载 User，校验密码与 banned 标记，通过后签发 JWT（包含 userId、username、role 等声明）。前端 LoginView 成功后将 token、user 写入 localStorage；router 可在 meta.requiresAuth 的路由上校验本地会话。

内容与多媒体。发帖 POST /api/content/publish：请求体含正文、作者 ID 和可选 imageUrl。若含图片，前端先 POST /api/upload multipart，后端存储至映射目录并返回可访问 URL，再随正文一并提交。服务端持久化 Content 后解析正文中的 #话题 标签并写入 content_tags；随后可触发 AIService 生成语义标签，失败不影响发帖成功返回。

互动与行为上报。点赞 POST /api/behavior/like、浏览 POST /api/behavior/view、点踩 POST /api/behavior/dislike、快速滑过 POST /api/behavior/skip：服务端写入 behaviors，并在点赞、点踩等场景同步维护内容计数。点赞与点踩互斥逻辑在 BehaviorService 中保证状态一致。搜索 GET /api/search：增加 userId 查询参数后写入 type=SEARCH 行为，以落实 SEARCH 类型行为采集。

兰州理工大学毕业设计

评论与转发。评论本质是父内容字段（parentContent）非空的内容实体（Content）；转发来源字段（repostOf）和引用来源字段（quoteOf）指向原帖 ID，列表接口通过这些字段区分渲染样式，减少多套 CRUD 模板。

【截图预留：放置登录成功后的首页信息流截图，可同时体现 For You/Following 标签、帖子卡片、用户侧边栏等核心入口。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、按钮和关键数据清晰可辨。】

图 4-1 系统登录与首页信息流界面

【截图预留：放置发帖弹窗或发布页、点赞评论转发交互、通知列表等截图，用于对应用户认证与内容互动实现。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、按钮和关键数据清晰可辨。】

图 4-2 内容发布、互动与通知功能界面

4.4 推荐排序实现

推荐服务是系统实现的重点。前端信息流列表组件（FeedList）请求 GET /api/content/feed?personalized=true，并在 Axios 拦截器中附带 Authorization。推荐服务（RecommendationService）负责组装候选：查询关注用户近期发帖作为网内候选（In-Network），合并全站候选作为网外候选（Out-of-Network），按内容标识（contentId）去重；随后调用负反馈服务（NegativeSignalService）完成过滤，再通过推荐策略管理器（RecommendationStrategyManager）的 getActiveStrategy() 获取当前策略。

传统混合推荐策略的带评分推荐方法（HybridRecommendationStrategy.recommendWithScore）流程如下：先调用协同过滤服务方法（CollaborativeFilteringService.getCollaborativeRecommendations）得到候选 ID 集；再对每条内容实体（Content）计算带明细的推荐分（calculateScoreWithDetails），填充评分拆分对象（ScoreBreakdown，含协同过滤加成字段）；最后按分数排序并经加权洗牌返回列表。管理员切换策略后无需重启进程，下一次请求即生效。

算法对比页面（CompareView）调用 /api/compare/feed、/api/compare/profiles、/api/compare/tuned 等接口：/feed 用于对比推荐与时间序，/profiles 用于展示不同用户画像和 Top 推荐，/tuned 用于在权重调节组件（WeightTuner）调节权重后重新拉取排序，体现

兰州理工大学毕业设计

参数敏感性实验。Feed 页在 For You 标签旁展示“传统算法”或“AI 推荐”徽标；AI 模式下缓存命中时直接返回 AI 排序，缓存未命中时先返回传统排序并在后台异步预计算 AI 结果。

AI 推荐策略通过 Prompt 调用 Ollama（如 `application.properties` 中 `ollama.rec.model`）。Prompt 含画像摘要与候选帖子摘要，并约束模型输出包含 `ranking` 与 `reasons` 的 JSON；解析成功后系统按 AI 返回顺序重排候选并写入推荐理由，解析失败或模型不可用时则在服务端降级为基础互动排序。策略管理接口同时返回 `aiAvailable` 与缓存统计，便于管理端判断 AI 服务状态。

【截图预留：放置个性化推荐 Feed、推荐策略徽标、推荐原因或评分拆分面板截图，突出推荐结果的可解释性。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、按钮和关键数据清晰可辨。】

图 4-3 个性化推荐信息流与评分明细界面

【截图预留：放置 CompareView、推荐流与时间序对比、权重调节组件 WeightTuner 或排序管道可视化截图。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、按钮和关键数据清晰可辨。】

图 4-4 推荐效果对比与权重调节界面

4.5 AI 模块与管理后台实现

AI 模块主要包括三类能力：自动打标使用较轻量模型（如 `ollama.model`）在发帖后异步执行，避免阻塞主链路；AI 推荐使用较大上下文模型（如 `ollama.rec.model`）专注排序决策；Grok 助手通过 DeepSeek API 提供对话，前端 GrokView 以聊天窗口呈现，与推荐链路解耦。

AdminView 集成统计卡片、用户表格、推荐策略单选卡片与 Ollama 状态展示：页面通过 `GET /api/admin/rec-strategy` 获取当前策略、AI 可用状态和缓存统计，并通过 `PUT /api/admin/rec-strategy` 切换 `traditional` 或 `ai`。批量打标接口遍历内容集合调用模型写入标签，用于历史数据补全。数据导入区域通过 `/api/admin/import-kaggle-batch` 或 `/api/admin/import-kaggle` 导入公开数据集，并在 Kaggle 不可用或数量不足时使用内置内容库补齐，以增强 Out-of-Network 候选多样性。

【截图预留：放置管理后台中 `traditional/ai` 策略切换、AI 可用状态、缓存统计、Kaggle 数据

兰州理工大学毕业设计

导入或批量打标界面截图。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、按钮和关键数据清晰可辨。】

图 4-5 管理后台推荐策略切换与数据导入界面

【截图预留：放置 AI 助手对话、用户画像详情、兴趣标签或画像可视化截图，用于说明 AI 模块与画像展示效果。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、按钮和关键数据清晰可辨。】

图 4-6 AI 助手与用户画像详情界面

4.6 广告智能分发实现

广告模块采用信息流原生广告形式。管理员在广告报表页（AdDashboard）或后台表单创建广告实体，配置定向标签（targetTags）与出价（bidPrice）。首页信息流列表组件（FeedList）拉取帖子列表后，根据全局广告间隔配置（interval）计算插入下标；每条广告展示调用统计接口增加展示次数（impressionCount），点击跳转前记录点击次数（clickCount），点击率（CTR）按式（4-1）计算，并动态进入质量分（qualityScore）。

$$CTR = clickCount / impressionCount \quad (4-1)$$

广告得分由标签匹配、出价和历史质量共同决定，可表示为：

$$qualityScore = CTR \times 100 \quad (4-2)$$

$$adScore = matchedTagCount \times 30 + bidPrice \times 10 + qualityScore \quad (4-3)$$

其中匹配标签数（matchedTagCount）为用户画像高频标签与广告定向标签（targetTags）的交集规模。该线性加权实现简单可解释，便于分析参数影响；后续可改为 Logistic 组合或加入频次惩罚。

广告业务实现补充。典型路径为：管理员在广告报表页（AdDashboard）中执行创建（POST）与编辑（PUT），并通过更新启用状态字段（active）控制广告启停；列表页（GET）展示广告明细与统计数据。首页展示前再 GET /api/ads/relevant 或等价接口拉取当前用户最相关广告，与帖子流在客户端按广告间隔（interval）合并。展示、点击分别调用统计接口回写展示次数（impressionCount）和点击次数（clickCount）。本系统以用户画像、广告标签和互动统计为基础实现规则排序，属于个性化广告推荐系统在毕业设

计规模下的工程化实现[3]。

【截图预留：放置首页原生广告卡片与广告报表 AdDashboard 截图，体现广告展示、点击统计、标签定向和启停管理。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、按钮和关键数据清晰可辨。】

图 4-7 信息流广告展示与广告报表界面

第 5 章 系统测试及使用说明

5.1 系统测试报告

5.1.1 测试的目的及原则

系统测试的目的是验证系统功能链路、推荐排序效果、数据导入与管理后台等核心模块是否满足任务书要求，并检查前后端联调过程中的接口稳定性和页面可用性。测试遵循覆盖核心业务流程、突出推荐算法效果、兼顾正常路径与异常路径的原则，测试证据主要包括页面运行结果、接口响应、构建输出和推荐对比页面截图。

5.1.2 测试过程及结果

系统测试采用“功能场景测试、接口联调验证、构建验证、推荐对比验证”相结合的方式。考虑到当前项目尚未建立独立的后端单元测试目录和前端组件测试脚本，本文不将测试结论表述为持续集成自动通过，而以手工执行记录、接口响应结果、页面运行截图和构建命令输出作为验证依据。功能测试围绕“注册登录—发帖—互动—通知/私信—推荐浏览—后台治理—广告统计”的端到端链路展开；推荐算法测试采用控制变量法，固定候选集合与用户画像，只改变单一因子观察评分拆分对象（ScoreBreakdown）或排序变化；构建验证则通过后端启动或依赖装配检查、前端 npm run build 结果确认项目在当前环境下可编译运行。

表 5-1 主要功能测试用例

编号	测试模块	测试内容	预期结果	结果
T01	注册登录	输入用户名和密码注册	注册成功并返回用户信息和 Token	通过

兰州理工大学毕业设计

T02	登录	使用正确账号登录	登录成功并进入首页	通过
T03	发帖	输入文字发布帖子	首页出现新帖子	通过
T04	图片发帖	上传图片后发布	图片地址保存并展示	通过
T05	评论	对帖子发表评论	评论数增加并生成通知	通过
T06	转发引用	转发或引用帖子	生成关联帖并更新计数	通过
T07	点赞点踩	点赞或点踩帖子	计数变化并记录行为	通过
T08	关注	关注其他用户	关注状态变为已关注	通过
T09	推荐流	访问个性化推荐	返回带排序结果的信息流	通过
T10	关注流	切换到 Following	仅展示已关注作者帖子	通过
T11	搜索	搜索关键词并带 userId	返回多类型结果并记录 SEARCH	通过
T12	热门话题	查看侧边栏热门话题	展示按互动聚合的标签列表	通过
T13	通知	点赞或评论他人帖子	对方收到通知并可标记已读	通过
T14	私信	发送私信	会话双方实时收到消息	通过
T15	用户画像	打开个人画像页	展示词云、雷达图与偏好统计	通过
T16	算法对比	打开 CompareView	展示推荐与时间序对比指标	通过
T17	参数调节	打开权重调节组件 (WeightTuner) 调权	调用 /api/compare/tuned 后指标变化	通过
T18	管理后台	切换推荐策略	策略切换成功且 Feed 徽标更新	通过
T19	AI 模式	Ollama 在线时切换 AI	返回 AI 排序或推荐理由	通过
T20	AI 降级	关闭 Ollama 后请求 AI	降级为基础互动排序	通过
T21	Kaggle 导入	管理员批量导入公	内容库新增帖子	通过

兰州理工大学毕业设计

		开数据集帖文	或由内置内容库补齐	
T22	广告	浏览首页带广告配置	按间隔插入广告并可点击统计	通过
T23	角色权限	普通用户访问后台接口	返回拒绝或前端隐藏入口	通过
T24	构建验证	mvn test 与 npm run build	构建成功无报错	通过

表 5-2 推荐算法测试项

测试项	测试方法	预期效果
标签匹配	用户点赞某类内容后刷新推荐	同类标签内容得分提高
时间衰减	对比新旧内容得分	新内容获得更高新鲜度优势
负反馈过滤	对作者屏蔽或对内容不感兴趣	对应内容不再进入推荐候选
协同过滤	构造相似用户行为	相似用户喜欢的内容获得加成
作者多样性	候选集中存在同作者多篇内容	同作者后续内容被适当降权
推荐对比	推荐流和时间流并排展示	推荐流平均分和标签命中率更高
随机探索	多次刷新推荐列表	列表顺序存在可控随机差异
策略切换	Admin 切换 traditional/ai	排序结果与徽标同步变化

为增强测试结论的可核验性，后续整理定稿材料时应在测试用例表之后补充运行截图，如图 5-1 至图 5-3 所示，分别记录功能用例执行、后端接口联调与前端构建/推荐对比页面运行情况。

【截图预留：放置注册登录、发帖、点赞评论、通知或后台管理等核心功能测试的页面执行截图。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、接口状态或命令输出清晰可辨。】

图 5-1 功能测试用例执行结果截图

【截图预留：放置后端服务启动日志、Postman/Apifox/curl 接口响应、推荐接口或管理接口联调结果截图。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、接口状态或命令输出清晰可辨。】

图 5-2 后端服务启动与接口联调结果截图

【截图预留：放置 npm run build 成功输出、CompareView 推荐对比、WeightTuner 调参或排

序管道漏斗图运行截图。后续将此处替换为系统实际截图，建议宽度 13~15 cm，并保证界面文字、接口状态或命令输出清晰可辨。】

图 5-3 前端构建与推荐对比页面运行截图

5.1.3 测试总结

功能测试整体通过率较高，说明核心业务闭环完整。推荐相关用例（T09–T20）验证了系统的可解释性与鲁棒性：评分拆分对象（ScoreBreakdown）与对比页能够呈现各分项贡献；AI 降级用例表明在推理服务不可用或超时等条件下仍能返回可用列表。

从对比指标角度，推荐流相对时间倒序在平均推荐分、标签命中率、TF-IDF 相似度均值上通常更优，但个别场景可能出现“过度个性化”导致话题收窄，此时可通过权重调节组件（WeightTuner）降低个性化权重，或提高探索随机扰动项（jitter）缓解。协同过滤与搜索行为（SEARCH）记录互补：前者挖掘被动曝光兴趣，后者捕捉主动意图，二者共同提升画像准确度。

构建验证只能说明依赖版本、编译配置和前端打包流程在当前环境下可用，不能替代自动化单元测试和压力测试。因此，本文将构建结果作为运行可行性的证据，将功能用例表、接口响应截图和推荐对比页面截图作为业务正确性的主要依据。当前不足在于尚未形成后端单元测试、前端组件测试、自动化 UI 测试和并发压力测试，难以量化高并发场景下的接口延迟，这部分在后续展望中进一步说明。

进一步从实验设计角度补充说明：当对比推荐流与时间倒序流时，应确保二者在同一时间窗口内抓取候选集合，否则“平均互动量”等指标可能因候选偏差而失真；使用权重调节组件（WeightTuner）调参时应记录前后权重向量，以保证图表与文字描述一致。作者多样性测试可在候选池人为注入同一作者多条帖子以放大惩罚效应；协同过滤测试则需预先构造相似用户行为簇，否则稀疏交互下加成可能不明显。

关于 AI 测试，除在线与离线两类场景外，还应记录 Prompt 长度与候选条数对耗时的影响：候选摘要过多会增加 Token 消耗与推理时间，过少则可能丢失语义上下文。测试可将候选限制在约二十条作为折中。实验记录应包含所用模型版本号与 temperature 配置，并与 application.properties 保持一致（可与 4.9 节测试数据与实验说明一并保存）。

构建与联调问题回顾。后端运行验证应优先确认 rec_db 可连接、application.properties

兰州理工大学毕业设计

指向本机环境，并通过服务启动日志和接口请求确认路由可用；若使用 `mvn -q test` 或打包命令作为构建校验，应说明当前结果主要反映编译与依赖装配状态，并不能替代独立单元测试。前端 `npm run build` 若因 Node 版本偏低或依赖冲突失败，可删除 `node_modules` 后按锁文件重装。跨域与 404 多由 Vite 代理未指向 8888 或 API 基地址写错导致，应优先对照浏览器网络面板与后端访问日志。此类问题属于工程配置与集成范畴，与算法正确性应区分排查；数据密集型系统设计也强调从数据存储、接口调用到运行维护的整体一致性[13]。

5.1.4 测试数据构造与实验建议

推荐算法输出对数据分布敏感。若帖子题材单一或行为稀疏，协同过滤与 TF-IDF 都难以体现差异。为此可在验证前构造分层数据集：用户层划分若干兴趣 `persona`，使其点赞与搜索关键词集中于不同标签簇；内容层为各簇准备带话题标签、长短不一的帖子，以触发深度匹配与新鲜度偏好差异；行为层安排交叉互动，使协同过滤能获得非空相似邻居。

对照实验可固定随机种子或关闭抖动因子中的随机项，以便重复刷新时观察排序稳定性；开启抖动后再观察列表多样性。对比页面上记录两种策略下的平均分与命中率差异时，应保存运行截图作为实验记录附录。

针对搜索行为（SEARCH），可设计检索词与帖子标签对齐，验证画像中出现检索意图相关的权重增量；针对负反馈，可屏蔽某一作者后刷新 For You，确认该作者内容不再进入候选集合。

5.2 系统使用说明

5.2.1 系统简介

本系统是一套面向社交媒体场景的实时信息流排序与分发原型系统。普通用户可以完成注册登录、浏览 For You 推荐流和 Following 关注流、发布帖子、评论、点赞、转发、引用、关注、搜索、查看通知、发送私信和查看画像等操作；管理员可以进行用户治理、推荐策略切换、AI 批量打标、Kaggle 数据导入、广告管理和推荐效果验证。

5.2.2 系统运行环境

表 5-3 系统运行环境配置

类别	环境或工具	说明
后端环境	JDK 17、Maven、Spring Boot 3.2.1	提供 RESTful API、WebSocket 服务与业务逻辑
前端环境	Node.js、Vue 3、Vite、Element Plus	提供单页应用、管理后台和图表展示
数据库	MySQL	存储用户、内容、行为、画像、通知、私信和广告数据
AI 服务	Ollama、DeepSeek API	用于内容打标、AI 推荐和智能问答

5.2.3 系统操作说明

普通用户首先通过注册或登录进入系统，登录成功后可在首页切换推荐流和关注流。用户浏览、点赞、评论、转发、搜索、点踩等操作会被记录为行为事件，并参与后续画像更新与推荐排序。

内容发布时，用户填写正文并可上传图片；系统保存帖子后解析话题标签，并可通过 AI 打标服务补充语义标签。其他用户对帖子进行互动时，系统会写入通知记录并通过 WebSocket 推送未读提醒。

管理员进入后台后，可查看用户与平台统计信息，切换 traditional 或 ai 推荐策略，执行 Kaggle 数据导入和 AI 批量打标，并在广告报表页维护广告配置、查看展示次数和点击次数。

算法验证时，可进入推荐效果对比页面，查看推荐流与时间流差异、评分拆分、权重调节组件和排序管道可视化结果。若 AI 服务不可用，系统会回退到传统混合推荐策略，保证信息流主链路可用。

设计总结

工作总结

本文围绕“基于用户行为分析的实时信息流排序与分发系统的设计与开发”这一课题，完成了从需求分析、系统设计、数据库建模、核心算法设计、前后端实现到系统测试的完整开发过程。系统采用 Spring Boot 与 Vue 3 构建前后端分离架构，使用 MySQL 存储核心业务数据，使用 WebSocket 实现通知与私信实时推送，并通过 ECharts 展示用户画像、推荐对比和广告统计等可视化内容。

(1) 业务与数据层面。系统实现用户、内容、行为、社交关系、标签、通知、私信、广告等实体的统一建模，覆盖注册登录、发帖评论、点赞点踩、转发引用、关注搜索、实时通知与私信等主流社交链路。行为表支持浏览 (VIEW)、点赞 (LIKE)、评论 (COMMENT)、转发 (REPOST)、搜索 (SEARCH) 等多类型事件，为画像与推荐提供可计算的特征基础；内容表通过父内容字段 (parentContent)、转发来源字段 (repostOf)、引用来源字段 (quoteOf) 等在一套模型上表达帖子、评论与二次传播，降低冗余代码。

(2) 推荐与画像层面。混合推荐策略 (HybridRecommendationStrategy) 将互动分、互动率、关注加成、标签匹配、TF-IDF 相似度、热门话题、协同过滤、内容深度、新鲜度、时间衰减、负反馈过滤、作者多样性惩罚与加权随机采样统一到同一打分管线；推荐策略管理器 (RecommendationStrategyManager) 支持与传统算法并行存在的 AI 推荐策略 (AiRecommendationStrategy)，实现策略热切换与降级排序。用户画像服务输出动态权重集合 (DynamicWeights)，根据行为数量与偏好分布自适应调节各分项权重，缓解冷启动与兴趣漂移问题。

(3) AI 与后台治理层面。AI 打标与 AI 推荐共享 Ollama 基础设施但模型规格分离，兼顾异步标注延迟与推荐推理质量；DeepSeek API 驱动的助手模块与排序链路解耦，避免对话流量冲击核心接口。AdminView 集成用户治理、策略切换、批量打标与外部数据导入，为对照实验提供可控输入。

(4) 评测与可视化层面。算法对比页面 (CompareView) 将对比指标、漏斗图与权

兰州理工大学毕业设计

重调节组件（WeightTuner）联结起来，使算法行为可通过调节权重观察指标变化；Feed 页徽标提示当前策略，便于区分实验条件。

（5）广告与商业化扩展层面。原生广告按间隔插入，匹配函数结合标签交集、出价与 CTR，展示点击统计闭环，为后续接入竞价与频次控制预留字段空间。

综上，本课题在工程完整性与算法表达之间取得了平衡：既能在有限时间内交付可运行系统，又能通过评分明细与对比实验支撑论文的技术论述。

不足与展望

由于时间与测试数据规模有限，系统仍存在一些不足。首先，当前推荐算法主要基于模拟数据和中小规模数据进行测试，尚未在大规模真实用户数据上验证效果。其次，系统的权限控制和安全校验仍有进一步加强空间，正式部署时应完善密码加密、接口鉴权、角色拦截和敏感操作审计。再次，AI 推荐策略依赖本地模型运行状态，模型推理速度和输出稳定性会影响推荐接口响应时间，需要结合缓存、异步任务和超时降级机制进一步优化。

未来系统可以从以下方向继续完善。第一，引入更大规模的数据集，优化召回层和排序层结构，形成更接近工业推荐系统的多阶段架构。第二，增加推荐解释能力，使用户能够了解推荐原因并主动调整兴趣偏好。第三，完善内容审核与反作弊机制，降低低质量内容和异常行为对推荐结果的影响。第四，进一步优化广告投放策略，将预算控制、频次控制和转化率预估纳入广告排序。第五，采用缓存、消息队列、异步任务和分布式部署提升系统并发能力。通过这些改进，系统可从业态原型逐步扩展为更完整、更稳定的社交媒体推荐平台。

从评测方法论角度，本文以离线指标与对照实验为主；后续可引入在线 A/B 测试框架：将用户随机分流到不同策略桶，观察留存、停留时长与互动率的显著性检验结果；离线侧则可构建训练集与验证集分离的行为日志重放，评估 NDCG、MAP、覆盖率与多样性指标，以降低单次实验路径依赖带来的偏差。与此同时，应记录模型版本、特征词典与权重快照，保证实验可复现。

从伦理与合规角度，推荐系统直接影响用户接收的信息结构，应在产品层面提供“关

兰州理工大学毕业设计

闭个性化”“只看关注”“减少某类话题”之类的可控入口；日志采集需最小化原则，仅保存完成任务所需字段，并对敏感字段脱敏存储。对于广告推荐混合场景，应明确区分标注社交内容与 sponsored 内容，防止用户误判信息来源，这一点亦是主流平台用户体验规范的重要组成部分。

个人收获

通过本课题，笔者将面向对象设计、数据库范式、Web 开发与机器学习基础串联为贯通的需求—设计—实现链条：从需求映射到数据字段，从接口契约驱动前后端协同，从评分拆分对象（ScoreBreakdown）审视推荐结果的可解释性与健壮性。与此同时，也体会到推荐系统既依赖模型与公式，也依赖数据治理与异常处理策略。后续将在软件质量与算法评测方向继续积累。

系统功能验证流程概要

为检验系统端到端行为一致性，可按下列顺序组织功能验证：用户注册登录后进入首页，分别访问个性化推荐流与关注流，确认列表加载与策略徽标展示正常；执行发帖、话题标签解析与图片上传，校验内容出现在预期信息流路径；打开用户画像页，核对词云、雷达图与统计字段与行为记录一致；进入算法对比页，读取推荐与时间序并列指标及漏斗图，并通过权重调节组件（WeightTuner）调整权重后复查 /api/compare/tuned 输出；在管理后台切换推荐策略（传统混合与 AI），确认 Feed 排序与接口返回随之变化；在 AI 策略下验证缓存命中、缓存未命中回退传统排序以及后台预计算路径；最后验证信息流广告按间隔插入及展示、点击统计闭环。上述流程覆盖核心业务链路的关键异常分支，可作为系统验收与回归测试的检查清单。

参考文献

- [1] 夏丽云, 徐敏赞, 丁懿楠, 等. 智能推荐算法下的科技期刊国际传播策略研究[J]. 中国科技期刊研究, 2023, 34(11): 1486-1493.
- [2] 付峻宇, 朱小栋, 陈晨. 基于图卷积的双通道协同过滤推荐算法[J]. 计算机应用研究, 2023, 40(1): 129-135.

兰州理工大学毕业设计

- [3] 张玉洁, 董政, 孟祥武. 个性化广告推荐系统及其应用研究[J]. 计算机学报, 2021, 44(3): 531-563.
- [4] 欧朝荣, 胡军. 融合显隐式反馈的协同过滤推荐模型[J]. 控制与决策, 2024, 39(3): 1048-1056.
- [5] Pei C, Zhang Y, Zhang Y, et al. Personalized re-ranking for recommendation[C]//Proceedings of the 13th ACM conference on recommender systems. 2019: 3-11.
- [6] 吴昌政. 基于前后端分离技术的 web 开发框架设计[D]. 南京: 南京邮电大学, 2020.
- [7] 刘华真, 王巍, 谷壬倩, 等. 基于用户浏览行为的个性化推荐研究综述[J]. Application Research of Computers/Jisuanji Yingyong Yanjiu, 2021, 38(8).
- [8] 高广尚. 用户画像构建方法研究综述[J]. 数据分析与知识发现, 2019, 3(3): 25-35..
- [9] HORSTMANN C S. Java 核心技术·卷 I: 开发基础[M]. 林信良, 译. 第 12 版. 北京: 机械工业出版社, 2022.
- [10] 杨海民. Vue.js 3.0 企业级管理后台开发实战: 基于 Element Plus[M]. 北京: 电子工业出版社, 2022.
- [11] 王树森, 黎崎. 深度学习推荐系统[M]. 北京: 电子工业出版社, 2020.
- [12] FREEMAN E, ROBSON E. Head First 设计模式[M]. 张晓菲, 等译. 第 2 版. 北京: 中国电力出版社, 2022.
- [13] KLEPPMANN M. 数据密集型应用系统设计[M]. 赵健博等译. 北京: 中国电力出版社, 2018.
- [14] 张敏军, 华庆一, 贾伟, 等. 基于深度神经网络的个性化推荐系统研究[J]. 西南大学学报 (自然科学版), 2019, 41(11): 104-109.
- [15] Twitter Inc. Twitter's Recommendation Algorithm[EB/OL]. (2023-03-31) [2026-03-13]. <https://github.com/twitter/the-algorithm>.
- [16] KOREN Y, BELL R, VOLINSKY C. Matrix factorization techniques for recommender systems[J]. Computer, 2009, 42(8): 30-37.
- [17] LINDEN G, SMITH B, YORK J. Amazon.com recommendations: Item-to-item collaborative filtering[J]. IEEE Internet Computing, 2003, 7(1): 76-80.

兰州理工大学毕业设计

[18] GUO H, TANG R, YE Y, et al. DeepFM: A factorization-machine based neural network for CTR prediction[J/OL]. arXiv, 2017 [2026-03-13]. <https://arxiv.org/abs/1703.04247>.

[19] ZHOU G, MOU N, FAN Y, et al. Deep Interest Evolution Network for click-through rate prediction[C].In: Proceedings of the AAAI Conference on Artificial Intelligence. Honolulu: AAAI Press, 2019: 5941-5948.

[20] 刘文贤, 朱海威, 武浩. 基于流行度和质量偏好建模的去偏推荐系统[J]. 云南大学学报(自然科学版), 2025, 47(3): 523-532.

[21] COVINGTON P, ADAMS J, SARGIN E. Deep neural networks for YouTube recommendations[C].In: Proceedings of the 10th ACM Conference on Recommender Systems. New York: ACM, 2016: 191-198.

外文翻译

原文

(此处根据学院要求另行填入外文文献原文。)

译文

(此处根据学院要求另行填入外文文献翻译稿。)

附录 程序源代码

(此处可在定稿阶段选取后端推荐策略、画像构建、AI 推荐、广告分发或前端可视化组件的核心代码片段作为附录。)

致 谢

在本次毕业设计的完成过程中, 指导教师在选题方向、需求分析、系统设计、论文结构与写作规范等方面给予了持续指导, 使本人能够在有限时间内完成系统实现与论文撰写。每当遇到推荐策略切换、协同过滤接入或论文章节衔接等问题时, 老师的建议帮助本人理清优先级与表达逻辑。

感谢学院提供的实验与文献资源, 使本人能够查阅推荐系统、数据库与 Web 工程相

兰州理工大学毕业设计

关资料并规范引用。感谢同窗好友在联调测试、界面体验与文字校对中的反馈，使系统缺陷得以提前暴露。感谢家人在生活与精神上的支持，使本人能够专心投入毕业设计阶段。

特别感谢在数据库设计与接口调试阶段给予帮助的同学：前后端字段不一致、跨域与 WebSocket 连接异常等问题往往需要在短时间内协同排查，大家的耐心沟通让系统得以稳定运行。也感谢在论文排版与格式检查阶段提出意见的同伴，使参考文献格式、图表编号与章节层级更符合学院规范。

最后，感谢开源社区与各大技术文档作者：Spring、Vue、ECharts、MySQL 与 Ollama 等项目的文档降低了工程门槛，GitHub 上的开源讨论也为定位异常提供了线索。本人深知当前系统在安全性、性能与评测严谨性方面仍有诸多不足，恳请各位评审老师批评指正。